

Detaljan Izvestaj Projekta: Atoll (V3.4)

Ovaj dokument opisuje kompletan projekat: strukturu, ideju, tok izvršavanja, logiku igre i AI modul. Uključeno je i objašnjenje zbog čega računar na početku igra oko ostrva.

1. Cilj projekta

- 1 Projekat implementira igru Atoll na heksagonalnoj tabli.
- 2 Podržana su dva rezima: covek-covek i covek-racunar.
- 3 Racunar bira poteze pomocu minimax algoritma sa alfa-beta odsecanjem i heuristikom.
- 4 Pobeda se proverava pravilom dominacije obodom nad ostrvima.

2. Struktura fajlova

- 1 main.py: ulazna tacka programa, učitavanje parametara i pokretanje GUI-a.
- 2 geometry.py: geometrija table (axial koordinate, susedstvo, projekcija u piksele).
- 3 state.py: model stanja igre i sva pravila (legalni potezi, pobeda, kraj igre).
- 4 ai.py: AI logika (kandidati, heuristika, minimax + alfa-beta).
- 5 gui.py: prikaz table, obrada klikova i game-loop sa automatskim AI potezom.

3. Tok izvršavanja programa

- 1 Program kreće iz main().
- 2 Unose se: n, rezim igre, igrač covek, dubina AI i limit kandidata.
- 3 Kreira se GameState i postavlja igrač koji počinje.
- 4 Pokreće se AtollGUI.run() petlja.
- 5 U petlji: obrada eventa -> eventualni AI potez -> redraw.
- 6 Nakon svakog poteza stanje proverava kraj igre.

4. Metode i ideja implementacije

4.1 main.py

- `_read_n()`: čita veličinu table i daje bezbedan default.
- `_read_first_player()`: definiše ko igra prvi potez.
- `_read_mode()`: bira rezim HH ili HAI.
- `_read_human_player()`: bira da li covek igra X ili O.
- `_read_ai_depth()`: bira dubinu pretrage AI-a.
- `_read_ai_move_limit()`: bira max broj kandidata po cvoru.
- `main()`: spaja sve korake, kreira state i pokreće GUI.

4.2 geometry.py

- `to_pixel(x, y, cx, cy)`: mapira axial polje u 2D ekran.

- NEIGHBORS: lista 6 smerova susedstva u heks mrezi.

4.3 state.py

- `_cube_z()`: racuna trecu cube koordinatu za geometriju.
- `_is_corner()`: proverava da li je polje teme heksagona.
- `GameState.__init__()`: postavlja pocetne strukture i kreira tablu.
- `_init_nodes_and_islands()`: gradi tablu, blokira coskove, postavlja pocetna ostrva.
- `_neighbors()`: vraca legalne susede jednog polja.
- `_edge_cells_excluding_corners()`: vraca obod bez coskova.
- `_axial_to_cartesian()`: pomocna transformacija za sortiranje po uglu.
- `_discover_islands_clockwise()`: pronalazi 12 ostrva i dodeljuje ID 0..11.
- `is_valid_move()`: proverava legalnost poteza.
- `get_valid_moves()`: vraca sva prazna polja.
- `clone()`: pravi kopiju stanja za AI simulacije.
- `apply_move()`: vraca novo stanje posle simuliranog poteza.
- `successors()`: vraca sve sledbenike (potez, stanje).
- `place_stone()`: menja realno stanje tokom partije.
- `_player_cells()`: sva polja igraca.
- `_connected_island_components()`: BFS komponenti ostrva igraca.
- `_count_to_last_in_dir()`: obodni koraci CW/CCW do poslednjeg ostrva komponente.
- `_boundary_dominance_score()`: globalni min(min(CW,CCW)) preko komponente.
- `check_winner()`: proverava da li neki igrac ima dominaciju ≥ 7 .
- `_update_game_over()`: azurira `game_over/winner` i zatvara igru i kao neresenu kada nema vise poteza.

4.4 ai.py

- `opponent()`: vraca suprotnog igraca.
- `_occupied_cells()`: iterator kroz zauzeta polja.
- `winning_moves()`: nalazi poteze koji odmah pobjedjuju.
- `_best_component_features()`: kljucne metrike komponenti ostrva.
- `evaluate()`: heuristika stanja iz perspektive igraca.
- `candidate_moves()`: pametno suzavanje kandidata pre minimax-a.
- `local_score()` (unutra): brzo lokalno rangiranje kandidata.
- `alphabeta()`: minimax pretraga sa odsecanjem.
- `choose_best_move()`: glavni AI API, root win check + minimax + fallback.

4.5 gui.py

- `AtolGUI.__init__()`: inicijalizacija prozora, fontova i AI parametara.
- `_is_ai_turn()`: proverava da li je AI na potezu.
- `_maybe_ai_move()`: trazi i odigrava AI potez.

- `_draw_island_labels()`: prikaz ID i vlasnika ostrva.
- `draw()`: crtanje mreze, celija i status teksta.
- `handle_click()`: klik pretvara u potez coveka ako je legalan.
- `run()`: glavni pygame loop.

5. Zasto racunar na pocetku igra oko ostrva

- 1 U startu su zauzeta samo polja ostrva (obodna X/O polja).
- 2 `candidate_moves()` gradi poteze kao susede svih zauzetih polja, pa su kandidati blizu ostrva.
- 3 `local_score()` nagradjuje potez koji dodiruje svoje ostrvo (+8).
- 4 `local_score()` kaznjava dodir protivnickog ostrva (-7).
- 5 Dodatni bonus je ako je potez na obodu (+2).
- 6 Zbog ova tri signala AI prirodno bira poteze koji grade veze izmedju sopstvenih ostrva, umesto random poteza u centru.

6. Zakljucak

Implementacija je modularna i logicno podeljena. Model stanja je cist, pravilo pobede je explicitno, a AI deo koristi standardni i ispravan pristup za turn-based zero-sum igre. Ponasanje racunara na pocetku je posledica dizajna heuristike i izbora kandidata, ne slucajnost.